UNIVERSIDADE JORGE AMADO
CIÊNCIA DA COMPUTAÇÃO

KELVIN MOREIRA DOS SANTOS OLIVEIRA
MARCOS HENRIQUE GONÇALVES DA SILVA
WILLIAM ARAÚJO GONÇALVES DA SILVA

DIAGNÓSTICO E PRIMEIRA OTIMIZAÇÃO DE UM ALGORITMO DE DETECÇÃO DE DUPLICATAS

Relatório da Avaliação 2 (Av2) apresentado à disciplina Análise e Otimização de Sistemas, do curso de Ciência da Computação, como parte da avaliação do semestre.

Professor: Igor Gonzalez

Salvador - BA
2026

1 INTRODUÇÃO

Detectar elementos repetidos em uma lista é uma tarefa comum na validação de cadastros e na limpeza de dados. A solução mais direta compara cada elemento com todos os demais e deixa de funcionar quando a lista chega a alguns milhares de itens.

Este relatório diagnostica uma implementação em Python com essa característica. Medimos o tempo da versão original, localizamos o gargalo, aplicamos uma primeira otimização com tabela hash e comparamos as duas versões em listas de tamanho crescente. A análise de complexidade (Big-O) prevê como o tempo cresce com o tamanho da entrada, e as medições mostram se a previsão se confirma (CORMEN et al., 2009).

O grupo aplicou o mesmo princípio ao agrupamento de anagramas, como extensão do tema, e mediu as duas tarefas com o mesmo procedimento. O código está em src/duplicatas.py, src/anagramas.py e src/benchmark.py.

2 DESCRIÇÃO DO PROBLEMA

O problema consiste em decidir se uma sequência contém ao menos um valor repetido. A entrada é uma lista de n elementos comparáveis por igualdade e, na versão otimizada, *hashable* (inteiros, textos, tuplas). A saída é um valor lógico: True quando existem duas posições com valores iguais e False caso contrário.

A versão original percorre todos os pares de posições (i, j) com $j > i$ e retorna True na primeira igualdade. Ela não usa memória auxiliar e aceita qualquer elemento comparável, inclusive listas aninhadas. O limite dessa versão é o tempo, que cresce com o quadrado de n (Seção 6).

No agrupamento de anagramas, a entrada é uma lista de palavras e a saída é uma lista de grupos, cada um com as palavras formadas pelas mesmas letras em ordem diferente. A comparação ignora maiúsculas, acentos e pontuação, de modo que "Ação!" e "acao" têm a mesma base. A versão original compara cada palavra com o primeiro elemento de cada grupo já formado e tem a mesma limitação quadrática.

3 ANÁLISE DA IMPLEMENTAÇÃO ORIGINAL

3.1 Detecção de duplicatas

```
def tem_duplicata(lista):
    for i in range(len(lista)):
        for j in range(i + 1, len(lista)):
            if lista[i] == lista[j]:
                return True
    return False
```

O laço externo escolhe a posição i, e o laço interno compara lista[i] com cada posição seguinte. A indexação e a comparação de inteiros custam O(1) cada.

No pior caso a lista não tem duplicatas, e o código executa $(n - 1) + (n - 2) + ... + 1 = n(n - 1)/2$ comparações, o que resulta em $O(n^2)$. O melhor caso ocorre quando os dois primeiros elementos coincidem: a função retorna na primeira comparação, com custo O(1). O espaço auxiliar é O(1), pois o código guarda apenas os índices i e j.

3.2 Agrupamento de anagramas

Para cada palavra, o laço interno percorre os grupos existentes (for grupo in grupos). A cada visita, o código normaliza e ordena as letras da palavra e do representante do grupo, com custo $O(k \log k)$ para palavras de comprimento k. No pior caso cada palavra forma um grupo novo, e a palavra de índice i percorre i grupos antes de ser inserida. O total é $n(n-1)/2$ visitas, e o custo temporal fica em $O(n^2 \cdot k \log k)$. O espaço auxiliar é $O(n)$ para as listas de grupos, mais $O(k)$ para as chaves temporárias.

4 IDENTIFICAÇÃO DOS GARGALOS

Na detecção de duplicatas, o gargalo é o laço interno. Cada elemento é comparado com todos os seguintes, e a função não guarda nada do que já viu, de modo que o número de comparações cresce com n^2. As medições sustentam o diagnóstico: de $n = 500$ a $n = 16.000$, o tamanho da entrada aumentou 32 vezes e o tempo da versão original aumentou cerca de 1.040 vezes. Um crescimento quadrático prevê 1.024 vezes (32^2).

No agrupamento de anagramas, a leitura do código aponta dois pontos de custo. O primeiro é a busca linear pelos grupos, que produz o termo n^2. O segundo é o recálculo da chave: a normalização e a ordenação do mesmo representante se repetem a cada visita e multiplicam o custo de cada comparação por $k \log k$. As medições confirmam o crescimento quadrático (de $n = 100$ a $n = 1.600$, a entrada cresceu 16 vezes e o tempo, cerca de 277 vezes, contra 256 previstas), mas não separam a contribuição de cada ponto, porque esta etapa não usou um *profiler*.

Dois aspectos ficam fora do que medimos. O primeiro é o consumo de memória. O segundo é o comportamento com duplicatas nas primeiras posições: nesse caso a versão original termina cedo e o gargalo não aparece. Os benchmarks usam somente o pior caso.

5 PRIMEIRA OTIMIZAÇÃO

5.1 Duplicatas com conjunto

```
def tem_duplicata_hash(lista):
    vistos = set()
    for item in lista:
        if item in vistos:
            return True
        vistos.add(item)
    return False
```

A estratégia troca as comparações entre pares por uma consulta a um conjunto. O código percorre a lista uma única vez e, antes de inserir cada elemento em vistos, verifica se ele já está lá. Se estiver, a função retorna True na hora. A consulta e a inserção em um set custam O(1) em média (PYTHON SOFTWARE FOUNDATION, [s. d.]), porque o Python calcula o hash do elemento e acessa a posição correspondente. Com n consultas e até n inserções, o tempo esperado é $O(n)$.

O conjunto ocupa $O(n)$ de espaço quando todos os elementos são distintos, então a otimização troca memória por tempo. Duas restrições se aplicam. Os elementos precisam ser *hashable*, e a função levanta TypeError para listas aninhadas. Além disso, colisões de hash degradariam cada operação para $O(n)$ no pior caso teórico, situação que não ocorre com inteiros comuns.

5.2 Anagramas com chave normalizada

A versão otimizada define chave_anagrama, que normaliza a palavra, ordena as letras e as converte em tupla. Ela calcula essa chave uma única vez por palavra e usa um dicionário (dict) que associa cada chave ao seu grupo. Duas palavras são anagramas se, e somente se, têm a mesma chave, e por isso a busca linear pelos grupos desaparece. A ordenação das letras passa a dominar o custo: $O(k \log k)$ por palavra e $O(n \cdot k \log k)$ no total, com $O(n \cdot k)$ de espaço para chaves e grupos. Com $k = 8$, fixo no benchmark, o custo cresce linearmente com n.

O projeto inclui testes automatizados que comparam o resultado das duas versões em casos fixos e em entradas aleatórias, para as duas tarefas.

6 EXPERIMENTOS E RESULTADOS

A função medir_tempo, de src/benchmark.py, executa a função medida uma vez para aquecimento, repete a medição e reporta a mediana dos tempos obtidos com time.perf_counter (PYTHON SOFTWARE FOUNDATION, [s. d.]). A versão original foi medida em 3 repetições de uma execução. A otimizada, muito mais rápida, rodou 7 repetições de 10 execuções (duplicatas) ou de 5 execuções (anagramas), e o tempo de cada repetição foi dividido pelo número de execuções.

As entradas de duplicatas são permutações de 0 a $n - 1$ com semente 42, sem repetições, o que obriga a versão original a percorrer todos os pares. As entradas de anagramas são n palavras aleatórias de 8 letras minúsculas, com chance desprezível de formarem anagramas entre si. O ambiente de referência foi Python 3.14.8 em Linux 7.2.11-arch1-1, com processador Intel Core i5-1135G7 (8 CPUs).

6.1 Detecção de duplicatas

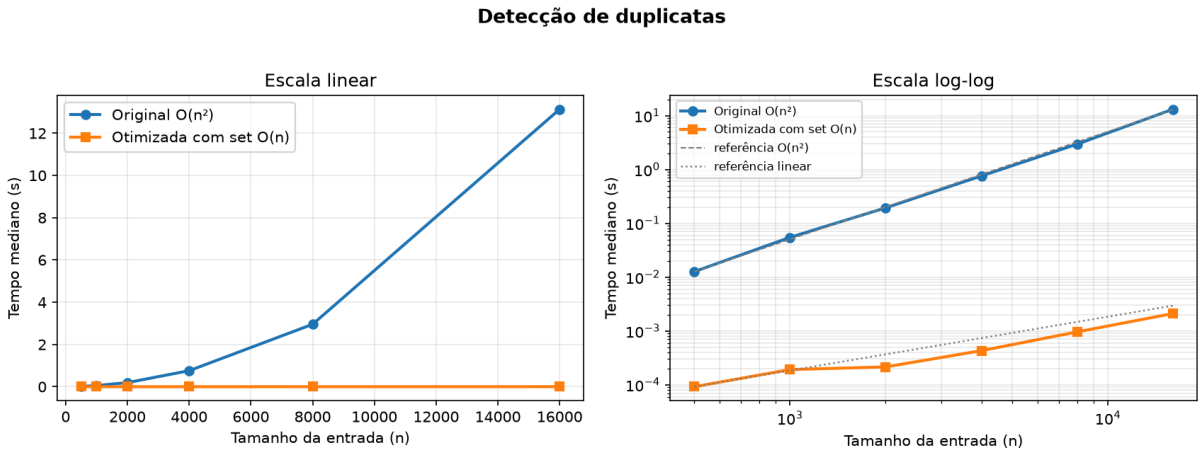
Tabela 1 – Tempo mediano da detecção de duplicatas no pior caso (sem duplicatas)

n	Original (ms)	Otimizada (ms)	Diferença (ms)	Ganho (vezes)
500	12,628	0,092	12,536	137,2
1.000	54,754	0,192	54,562	285,7
2.000	194,058	0,215	193,843	903,9
4.000	757,993	0,430	757,563	1.762,1
8.000	2.954,888	0,960	2.953,928	3.077,8
16.000	13.132,350	2,104	13.130,245	6.240,9

Fonte: Elaborado pelos autores (2026), com dados de benchmark_duplicatas.csv.

Com $n = 500$, a versão original levou 12,63 ms e a otimizada, 0,09 ms. Com $n = 16.000$, os tempos foram 13,13 s e 2,10 ms. A cada dobra de n, o tempo da original se multiplicou por 3,5 a 4,4, e o da otimizada, por 1,1 a 2,2. O valor de 1,1, entre 1.000 e 2.000, destoa do restante, e não investigamos sua causa. Entre o menor e o maior n, a inclinação média na escala log-log foi 2,00 para a original e 0,90 para a otimizada, próximas das inclinações 2 e 1 previstas pela análise (SEDGEWICK; WAYNE, 2011).

Figura 1 – Tempo mediano da detecção de duplicatas em função de n, em escala linear e log-log



Fonte: Elaborado pelos autores (2026).

O ganho aumentou de 137,2 vezes, com $n = 500$, para 6.240,9 vezes, com $n = 16.000$, e se multiplicou por 1,7 a 3,2 a cada dobra de n. Isso se alinha à teoria, pois a razão entre uma função quadrática e uma linear cresce em proporção a n. O ganho foi positivo em todos os tamanhos testados e mais expressivo nos maiores.

A análise assintótica descreve a taxa de crescimento, não o tempo absoluto. Os tempos medidos incluem as constantes do interpretador, e o set tem custo próprio de criação e de cálculo de hash. Como o menor tamanho testado foi $n = 500$, o experimento não mostra o ponto em que as constantes reduziriam ou inverteriam a vantagem da otimizada. Os resultados valem para o pior caso, em uma máquina, com uma semente. Também não medimos o melhor caso, o caso médio nem o uso de memória.

6.2 Agrupamento de anagramas

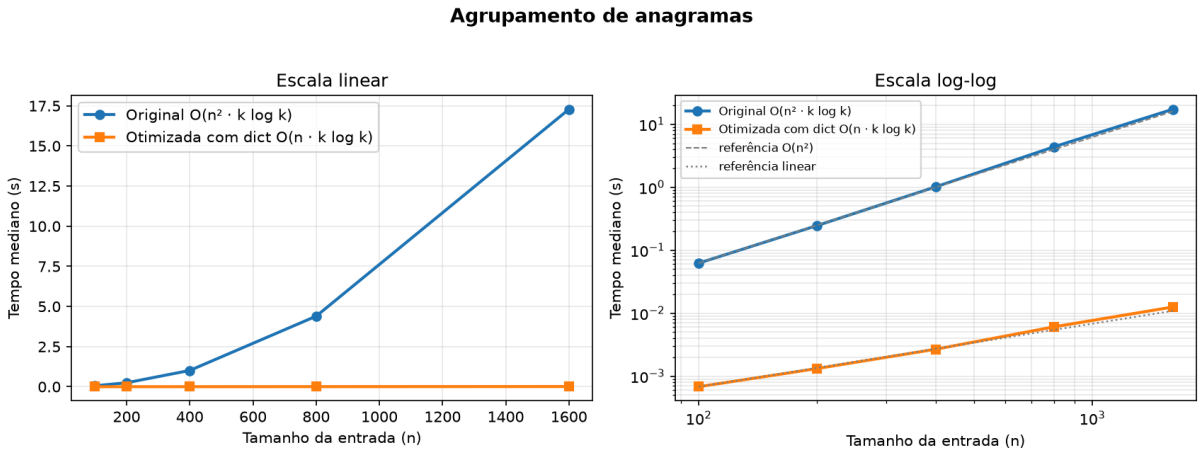
Tabela 2 – Tempo mediano do agrupamento de anagramas no pior caso (palavras com chaves distintas)

n	Original (ms)	Otimizada (ms)	Diferença (ms)	Ganho (vezes)
100	62,384	0,681	61,703	91,7
200	246,856	1,323	245,533	186,6
400	1.013,887	2,681	1.011,207	378,2
800	4.394,012	6,080	4.387,933	722,8
1.600	17.285,736	12,534	17.273,202	1.379,1

Fonte: Elaborado pelos autores (2026), com dados de benchmark_anagramas.csv.

O comportamento se repete. De $n = 100$ a $n = 1.600$, o tempo da original passou de 62,38 ms para 17,29 s, e o da otimizada, de 0,68 ms para 12,53 ms. O ganho subiu de 91,7 para 1.379,1 vezes. A cada dobra de n, a original se multiplicou por 3,9 a 4,3 e a otimizada, por 1,9 a 2,3, com inclinações log-log de 2,03 e 1,05. A otimizada continua pagando a normalização e a ordenação de cada palavra, custo que a mudança de estrutura não elimina.

Figura 2 – Tempo mediano do agrupamento de anagramas em função de n, em escala linear e log-log



Fonte: Elaborado pelos autores (2026).

7 CONSIDERAÇÕES FINAIS

A atividade pedia o diagnóstico de um algoritmo, uma primeira otimização e a medição do efeito. A detecção de duplicatas original compara todos os pares e custa O(n^2) no pior caso. O gargalo é o laço interno, que repete comparações sem guardar o que já viu. Substituir esse laço por um set reduziu o tempo esperado para O(n) e elevou o espaço auxiliar de O(1) para O(n). Nas medições, o ganho foi de 137 vezes com $n = 500$ e de 6.241 vezes com $n = 16.000$.

A mesma ideia, aplicada ao agrupamento de anagramas com uma chave normalizada e um dicionário, reduziu o custo de O($n^2 \cdot k \log k$) para O($n \cdot k \log k$) e produziu ganhos de 92 a 1.379 vezes.

O trabalho mostrou que a análise de complexidade prevê o formato das curvas medidas e que dobrar n é um teste simples para conferir essa previsão. Mostrou também que o ganho depende do cenário. Medimos apenas o pior caso, em uma máquina e com entradas sintéticas, e uma duplicata nas primeiras posições faz a versão original terminar cedo.

REFERÊNCIAS

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to algorithms*. 3. ed. Cambridge: MIT Press, 2009.

PYTHON SOFTWARE FOUNDATION. *Python 3 documentation*: set, dict, time.perf_counter, unicodedata. [S. l.]: Python Software Foundation, [s. d.]. Disponível em: https://docs.python.org/3/. Acesso em: 30 out. 2026.

SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. [S. l.]: Addison-Wesley, 2011.